

git-why

Production-readiness report

READY FOR RELEASE

A local, read-only Git history CLI

The release candidate answers one narrow question: which recorded Git history explains a line of code? It reads the local repository at HEAD, reports the last-changing commit and earlier evidence, and marks incomplete history instead of inventing an explanation.

22 / 22

local tests

3 / 3

hosted OS jobs

0

known OSV findings

PASS

remote install

Scope

- Package and binary: `git-why 0.1.0`; minimum Rust 1.85 and Git 2.15.
- Source under review: `b915793f63398afec4fa75c125cecd3f9c684307`.
- Public repository: github.com/whatnotbot/git-why.
- Decision covers the CLI, package, public documentation, continuous integration, and release workflow. No hosted service is part of this release.

REVIEW DATE	20 July 2026
HOSTED CI	Run #2: Ubuntu, macOS, and Windows passed
REMOTE PROOF	SSH main SHA matched; HTTPS Cargo install ran the pushed commit
REPORT NOTE	The later report-only commit changes no Rust source

Verification matrix

Every result below was produced from the repository or the pushed GitHub revision. The hosted matrix is linked in the evidence column.

CHECK	EVIDENCE	RESULT
Rust 1.85 gate	Format check; 6 unit tests; 16 integration tests; strict Clippy; release build; locked dependencies.	PASS
Current Rust gate	The same format, test, Clippy, and release commands on the installed stable toolchain.	PASS
GitHub Actions	CI run #2 completed on ubuntu-latest, macos-latest, and windows-latest.	PASS
Committed package	cargo package --locked built and verified the crate from a clean HEAD.	PASS
Remote install	cargo +1.85.0 install --git https://github.com/whatnotbot/git-why --locked installed commit b915793.	PASS
Git dispatch	Both git why Cargo.toml:1 and direct git-why execution returned valid evidence; --version returned 0.1.0.	PASS
CLI contract	README help, human output, and JSON examples were compared with the binary; JSON schema_version is 1.	PASS
Dependency audit	The complete locked all-target graph was queried against OSV; no known vulnerabilities were returned.	PASS
Release rehearsal	Native macOS arm64 archive and SHA-256 checksum were created; the archive contains the binary, README, and license.	PASS
Performance sample	On a repository with 1,294 tracked files and 1,847 commits: 0.35 s median; 0.53 s cold run.	PASS

The corrective loop

1	Initial hosted run	Linux and macOS passed. Windows failed during the wildcard-filename fixture.
2	Root cause	The test tried to create <code>literal*.rs</code> ; <code>*</code> is not a valid Windows filename. The product did not fail.
3	Fix	The Unix-only filename test was scoped with <code>cfg(not(windows))</code> . Other portable path tests remain active on Windows.
4	Rerun	CI run #2 completed successfully across all three operating systems.

Security and reliability

The command handles source text, commit metadata, paths, and Git output from repositories that may not be trusted. The review concentrated on containment, side effects, credential access, bounded input, and terminal safety.

Repository boundary	Lexical and canonical containment checks reject paths outside the worktree and reject every symbolic-link component.
Process boundary	Git is executed directly without a shell. Standard input is closed; credential prompts, lazy fetches, pagers, optional locks, external diffs, text conversion, file-system monitors, and signature display are disabled.
Network behavior	The program makes no HTTP requests. GitHub issue links are derived locally from a recognized origin URL.
Resource limits	Source blobs and Git stdout/stderr are capped at 5 MiB; commit metadata is capped at 1 MiB; reports contain at most six evidence commits.
Output safety	Human output removes terminal controls and bidirectional formatting characters. JSON is encoded through <code>serde_json</code> .
Repository state	The tool reads evidence from HEAD and never writes the repository. Dirty target files are reported and working-tree edits are excluded from the evidence.
Supply chain	Cargo.lock is committed. CI uses Rust 1.85.0 and pinned official GitHub Actions revisions with least-privilege permissions.

Known limits

- A pathological local history can make Git blame or line-history traversal take a long time; there is no wall-clock timeout.
- The installed Git executable and applicable system configuration remain part of the local trust boundary.
- Commit messages are evidence written by people; they can be incomplete or wrong. The tool does not infer intent from code.
- Shallow, partial, truncated, or fallback history is reported as incomplete. The tool does not fetch missing objects.
- Only tracked UTF-8 text at HEAD is supported. One line is accepted per command; ranges and symbol lookup are out of scope.
- The release workflow was reviewed and locally rehearsed. It will publish archives only when a matching version tag is intentionally pushed.

DECISION	These limits are documented and do not block the 0.1.0 local CLI release.
----------	--

Operator handoff

Install from the public repository over HTTPS:

```
cargo install --git https://github.com/whatnotbot/git-why --locked
```

Or clone over SSH and install from the checkout:

```
git clone git@github.com:whatnotbot/git-why.git
cd git-why
cargo install --path . --locked
```

Inspect a tracked line from anywhere inside the worktree:

```
git why src/auth.rs:42
git why --json src/auth.rs:42
```

Public documentation review

README commands, output, exit behavior, data access, privacy boundaries, and limits were checked against the binary. The supplied [Wikipedia writing-pattern page](#) was used as a descriptive checklist, not as an authorship detector. The prose avoids inflated importance claims, canned openings, vague attribution, faux quotations, excessive emphasis, and padded conclusion sections.

Release assets

Linux x86_64	git-why-v0.1.0-x86_64-unknown-linux-gnu.tar.gz
macOS Apple Silicon	git-why-v0.1.0-aarch64-apple-darwin.tar.gz
macOS Intel	git-why-v0.1.0-x86_64-apple-darwin.tar.gz
Windows x86_64	git-why-v0.1.0-x86_64-pc-windows-msvc.zip

Each release archive is paired with a SHA-256 checksum. The tag must match Cargo.toml before the workflow builds or publishes anything.

READY

Code, docs, package, remote install, and the hosted operating-system matrix have been verified. Version 0.1.0 can be tagged when the maintainer chooses to publish a release.